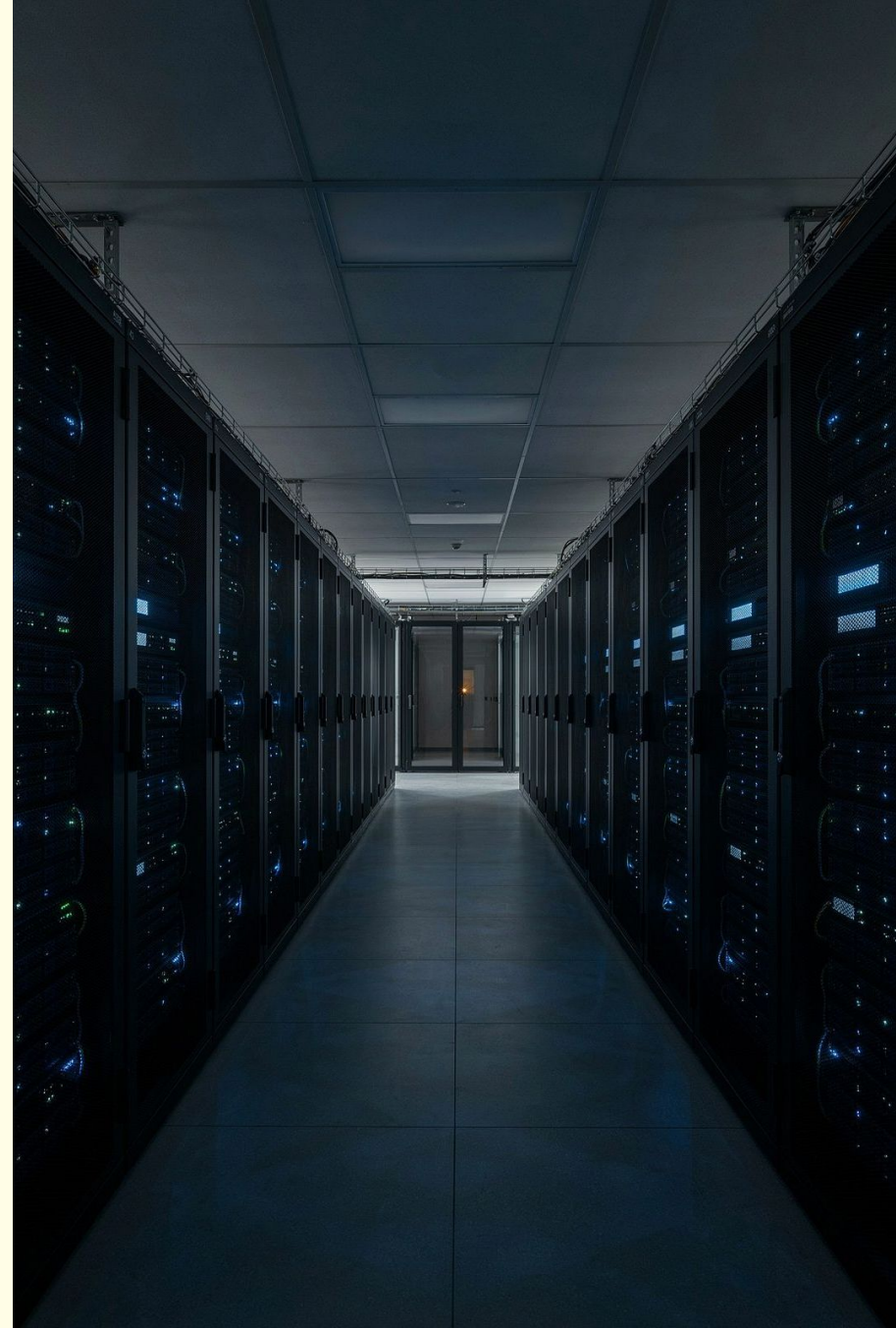


Context Engineering for Program Managers and Team Leads

How global AI teams build reliable assistants and agents — and what leaders should ask their teams. No code required. Every concept illustrated through one running example: an **internal HR policy assistant** that answers employee questions about leave, travel, and benefits.

TARKA UPSKILLING

50-SLIDE CONTENT PACK



What We Will Cover Today

This session builds from foundations to frameworks. Parts 1–3 build understanding. Part 4 gives you vocabulary your engineers already use. Parts 5 and 6 show real systems. Parts 7 and 8 are what you take back to your team.

01

Foundations

What the model actually receives – the eight parts of a request, tokens, and the context window

02

Why Context Fails

Four landmark research findings: Lost in the Middle, Context Rot, attention budgets, and multi-turn drift

03

The Four Core Moves

Write, Select, Compress, Isolate – the vocabulary engineering teams use every day

04

Case Studies

Manus, Anthropic, Cognition, Shopify, Google, Morgan Stanley, and ETH Zurich

05

Framework + Leader Questions

A 7-step context management framework and ten questions to ask your team on Monday

PART 1 · THE CLAIM

Context Engineering: Deciding What the AI Sees

In June 2025, a phrase began circulating across AI teams worldwide: **"prompt engineering is dead."** What replaced it was not a new technology — it was a clearer name for a job that had already outgrown its original description. Context engineering is the discipline of deciding *everything* the AI model receives, not just the wording of the question you type.

This session is for programme managers, team leads, and professionals who are new to AI but responsible for AI products. You will leave with a framework, a vocabulary, and a concrete list of questions to take to your engineering team.



"Prompt Engineering Is Dead" — Where This Claim Came From

The phrase did not appear in a research paper. It spread from a sequence of high-profile posts by practitioners who had built real AI products at scale. Understanding the sequence helps you separate the genuine insight from the hype.

June 2025 — Tobin Lütke, Shopify CEO

1

Introduced the phrase "context engineering": *"providing all the context for the task to be plausibly solvable."* He was not dismissing prompts — he was saying the word "prompt" makes the job sound smaller than it is.

July 28, 2025 — Gartner

3

Published formal guidance: *"Context engineering is in, and prompt engineering is out."* This is where the LinkedIn debate exploded. Gartner's framing was sharper than the practitioners intended.

2

One Week Later — Andrej Karpathy, OpenAI co-founder

Agreed publicly: context engineering is *"filling the context window with just the right information for the next step."* Same insight, different words: the problem is curation, not phrasing.

4

Late 2025 onward — Anthropic, Google, LangChain, Manus

All published engineering guides using the term. It entered the industry's technical vocabulary as a recognised discipline with defined techniques and measurable outcomes.

❏ The takeaway: these leaders did not say prompts stopped mattering. They said the word "prompt" makes the full job sound smaller than it is. In real products, the instruction is one small part of what the model receives.

The Correction: Prompt Engineering Became One Part of a Bigger Job

The LinkedIn debate created a false binary. The accurate picture is that prompt engineering is a subset of context engineering – still important, but no longer sufficient on its own. Here is the precise distinction:

Dimension	Prompt Engineering	Context Engineering
What it controls	The wording of the instruction	Everything the model receives: instructions, documents, chat history, tool results, memory, output rules
When it happens	Once, while writing the prompt	Every time the system builds a request – potentially thousands of times per day
Who does it	Anyone writing a prompt	The engineering team, through system design decisions
Typical question	"How should I phrase this?"	"What should be in this request, and what should stay out?"

Anthropic describes context engineering as the natural next step after prompt engineering – not its replacement. A well-crafted instruction still matters enormously. But when an assistant answers from company documents, remembers past conversations, and invokes tools, most of what the model reads was assembled by the system, not typed by the user.

The Model Starts From Zero on Every Request

This is the single most important fact to understand about language models. A model does not remember your earlier conversations on its own. Each time you or a user sends a message, the application assembles a fresh request from scratch and sends it to the model. The model reads the entire request, generates an answer, and retains nothing afterward.

If something is not in the request – company policy, the user's name, yesterday's chat, a tool result from two steps ago – the model does not know it exists. It cannot guess. It cannot look it up. It simply works with what it was given.

📌 **Key point for leaders:** When ChatGPT or Claude appears to "remember" you, the application is saving notes somewhere and inserting them into each new request. The model itself holds nothing between requests. This single fact is the reason context engineering exists: someone has to decide what goes into every request, every time.

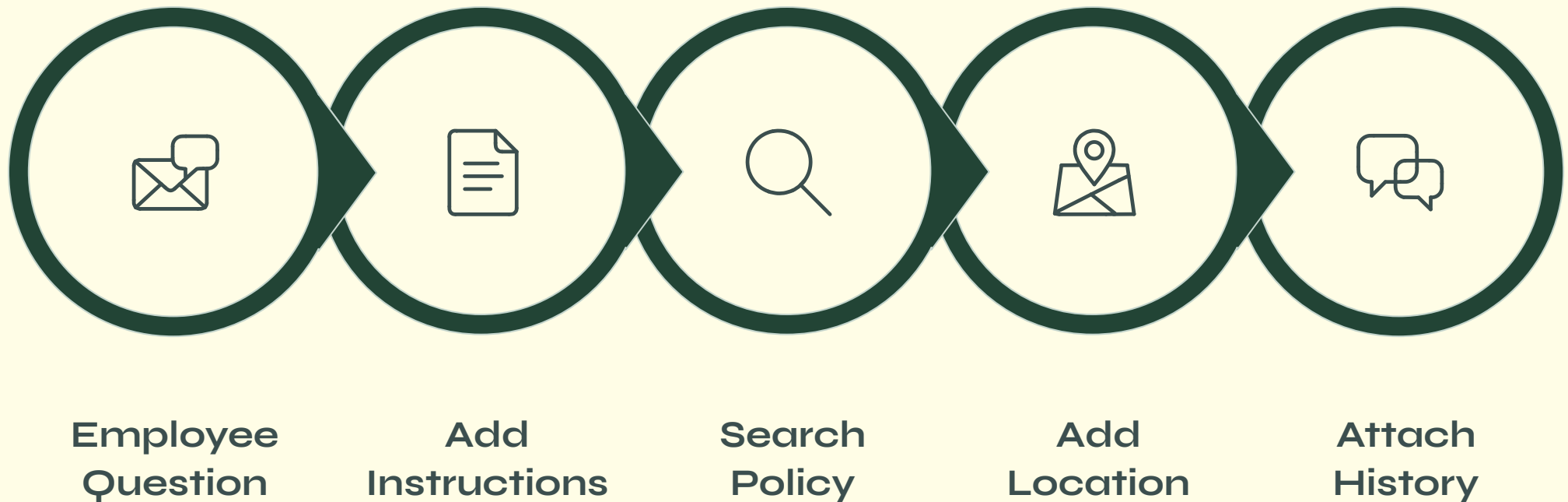
The Eight Parts of a Request

Every request a model receives is assembled from up to eight distinct components. Prompt engineering mainly addresses the first row. Context engineering addresses all eight – and the decisions about how they fit together inside one request. This is illustrated using the running HR policy assistant example throughout.

Part	What It Is	HR Assistant Example
System instructions	Fixed rules: role, tone, limits	"Answer only from HR policy. If unsure, say so."
User question	What the person typed	"How many casual leaves do I get?"
Conversation history	Earlier messages in this chat	Employee asked about sick leave two messages ago
Retrieved documents	Passages pulled from company files for this question	Leave Policy v3.2, section 4
Tool descriptions	Actions the model is allowed to request	"get_leave_balance", "raise_hr_ticket"
Tool results	Data returned after an action runs	"Balance: 6 casual leaves remaining"
Memory	Saved facts from earlier sessions	Employee works in the Pune office; prefers English
Output rules	Required answer format	"Under 100 words, cite the policy section"

One Request, Assembled Step by Step

Here is what actually happens when an employee types a single question to the HR assistant. Each arrow below represents a design decision your engineering team makes – and the last line is the most important one.



The user typed one sentence. The model received five or six blocks of information, carefully selected and ordered. If no passage scores above the relevance threshold, the assistant replies *"I could not find this in policy; please contact HR"* – rather than guessing. That fallback behaviour is also a design decision, and one of the most important ones your team will make.

Tokens: The Unit Everything Is Measured In

A token is a small piece of text the model reads – often a whole word, sometimes part of one. Tokens are the fundamental currency of AI systems: cost, speed, and context limits are all measured in them.

~75 words

≈ 100 tokens in English

Charged separately

Input tokens (what you send) and output tokens (what the model writes) are priced differently

100:1 ratio

In agents, Manus reports roughly 100 input tokens for every 1 output token

Why the 100:1 ratio matters for leaders

In agent products – systems that search, read files, and call tools across many steps – the answer the model writes is short. The request carrying all the gathered information is long. This means your cost is driven almost entirely by context design, not by the answers the model produces.

Every part of the request – instructions, documents, conversation history, tool results – uses tokens. Understanding token flow is the first step to controlling cost.

Source: Manus, "Context Engineering for AI Agents: Lessons from Building Manus" (Jul 2025)

Context Window: The Maximum One Request Can Hold

The context window is the maximum number of tokens a single request – plus its answer – can contain. If a request exceeds the limit, the system must cut or shorten something before sending. Leading models now advertise windows ranging from about 128,000 to over 1 million tokens.

One million tokens is equivalent to several long novels. It sounds like the problem of "running out of space" is solved. It is not – and the next slides explain exactly why.

Models do not use long requests well

Accuracy drops well before the advertised limit is reached. The usable window is consistently smaller than the marketed number, especially for tasks that require combining facts or following references.

Long requests cost more

Because providers charge per token, a request twice as long costs roughly twice as much – and agent tasks can grow to fifty times their initial size as tool calls accumulate.

Long requests run slower

The model must read the entire request before it writes a single word of the answer. A larger request means longer wait times before the first response appears – a significant experience problem in live products.

Advertised Size vs. Usable Size

NVIDIA's RULER benchmark tested 17 leading models on tasks harder than the simplest "find one fact in a long text" test. The findings should reshape how your team specifies model requirements.



All models claimed windows of 32,000 tokens or more

Only about half performed well at that length. Almost all fell below an acceptable quality bar before reaching their advertised limit.



Simple tasks hid the problem

Models that scored near-perfectly on "find one fact" tests dropped sharply when the task required combining facts from multiple places, comparing clauses, or following a chain of references – which is what real work looks like.



Leader takeaway

Ask what length your model was tested at *for your specific task type*, not the advertised number. Vendor benchmarks typically use the easiest possible test at the maximum window size.

Source: Hsieh et al., "RULER: What's the Real Context Size of Your Long-Context Language Models?" (NVIDIA, COLM 2024)

What Happens as the Window Fills Up

As a request grows longer, four compounding problems emerge simultaneously. In a simple chatbot answering one question, these rarely surface. In an agent that searches, reads files, and calls tools across fifty steps, all four hit at once.



Cost rises linearly

Every token in the request is billed. A task that runs 50 tool calls generates a request that grows fifty times over its starting size, multiplying the bill with each step.



Speed drops

Longer requests mean a longer wait before the first word of the answer appears. In user-facing products, this latency is felt immediately and reduces trust in the assistant.



Accuracy falls

Information placed in the middle of a long request is consistently used less well than information at the start or end. More tokens means more of your evidence lands in this danger zone.



Forced truncation

When the window is full, the system must drop or summarise content before sending. Done carelessly, a key fact from an earlier step disappears permanently from the model's view.

Source: Manus, "Context Engineering for AI Agents: Lessons from Building Manus" (Jul 2025). Manus reports approximately 50 tool calls per task, each expanding the request.

So What Exactly Is Context Engineering?

Anthropic's Definition

The strategies for choosing and maintaining the **best set of information** in each request – including everything outside the prompt itself.

Their guiding principle: find the "**smallest possible set of high-signal tokens**" that makes the desired result most likely.

Source: Anthropic, "Effective context engineering for AI agents" (Sep 2025)

Three decisions on every request

1 What goes in

Which documents, tool results, memory facts, and history items are included in this specific request

2 In what order

Where each piece of information is placed within the request – because position determines how well the model uses it

3 What stays out

"High-signal" means information that directly helps answer this question. Everything else is noise that costs money, consumes attention, and reduces accuracy

This is repeated work. The decision is made fresh every time a request is built – not once at the start of the project.

Research 1: "Lost in the Middle" – Stanford, 2023

This is one of the most replicated and practically important findings in applied AI research. Researchers placed the single document containing the correct answer at different positions within a set of 20 documents, then measured accuracy across GPT-3.5, GPT-4, Claude, and open models. The results were consistent and striking.

What they found: a U-shaped accuracy curve

Accuracy was highest when the relevant document was placed first or last in the set. It was lowest – by a large margin – when placed in the middle. Drops exceeded 30% in some configurations, using the same model, same documents, and same question.

What it means for your HR assistant

When your retrieval system returns four policy passages, the one most likely to contain the answer should not sit between three weaker passages. It should be first or last. This is a layout decision, not a model decision.

Takeaway for leaders

Order matters as much as content. Critical instructions and the strongest evidence go at the start or the end – never buried in the middle of a long request. Ask your team: how is the retrieved content ordered before it goes to the model?

Source: Liu et al., "Lost in the Middle: How Language Models Use Long Contexts" (TACL 2024, arXiv 2307.03172)

Research 2: "Context Rot" – Chroma, July 2025

This study directly tested whether longer inputs hurt performance independently of task difficulty. The team held the task constant and varied only the length of the input – testing 18 models including GPT-4.1, Claude 4, Gemini 2.5, and Qwen3.

Finding 1: every model degraded

Every model tested became less reliable as input length increased – even on simple tasks like finding or copying a specific piece of text. No model was immune.

Finding 2: the surprising shuffle result

Models performed *better* when surrounding text was shuffled nonsense than when it was realistic, well-structured, and topically related. Coherent-looking irrelevant text is more confusing than obvious gibberish.

Finding 3: "similar but wrong" is dangerous

Retrieving passages that look relevant but do not contain the answer is worse than retrieving nothing at all. Your retrieval quality threshold matters more than retrieval quantity.

⚠️ Takeaway for leaders: length alone hurts, independent of difficulty. And retrieving "similar but wrong" passages is worse than retrieving nothing. Ask your team: what happens when the search returns something that looks right but is not?

Source: Hong, Troynikov & Huber, "Context Rot: How Increasing Input Tokens Impacts LLM Performance" (Chroma, Jul 2025)

Research 3: The Attention Budget — Anthropic, 2025

The Mechanics

Inside the model, every token is compared with every other token to determine which pieces of information matter for generating the next word.

1,000 tokens → approximately 1 million comparisons

100,000 tokens → approximately 10 billion comparisons

As the request grows, the model's capacity to focus is spread across exponentially more comparisons. Models also saw far more short texts than very long ones during training, making very long requests inherently less familiar territory.

Why this unifies the other findings

The "Lost in the Middle" result and the "Context Rot" result both flow from the same underlying mechanism. Anthropic calls this the **attention budget**: a finite resource that every added token draws down.

The practical implication is not a sudden failure at a specific length — it is a *steady, continuous loss of precision* as the request grows. There is no safe length above which things suddenly break. Quality erodes gradually, which makes it hard to detect in casual testing.

Your team's job

To spend the model's attention budget only on information that genuinely helps answer the question at hand. Everything else is a tax on accuracy.

Source: Anthropic, "Effective context engineering for AI agents" (Sep 2025)

Research 4: Models Get Lost Across Many Turns – Microsoft & Salesforce, 2025

This large-scale study tested 15 models across more than 200,000 simulated conversations covering six task types. The core experimental design: give a model the same task all at once versus spread across several messages the way a real user would give them.

39%

Average performance drop

When requirements were spread across multiple messages rather than given in one complete instruction, models dropped by an average of 39% on performance metrics

15

Models tested

Across six task types, in 200,000+ simulated conversations – making this one of the most comprehensive studies of multi-turn AI behaviour

200K+

Conversations simulated

Scale and rigour that remove concerns about statistical noise – this is a robust, reproducible finding across model families

Main cause: models guessed early based on partial information, then kept building on their own wrong guess as more details arrived. Each new message compounded the error rather than correcting it. **Practical fix your team can apply today:** when requirements arrive in pieces across a conversation, the system should gather them into one clean summary before the model generates its final answer.

Source: Laban et al., "LLMs Get Lost in Multi-Turn Conversation" (Microsoft Research & Salesforce, arXiv 2505.06120, May 2025)

Four Ways Context Goes Wrong

Drew Breunig's widely cited 2025 analysis names four distinct failure modes. These names give your team a shared vocabulary for bug reports and post-mortems. Instead of "the bot gave a wrong answer," a tester can say "this is a clash failure" – which points directly to the type of fix needed.

Failure	What Happens	HR Assistant Example	Fix
Poisoning	A wrong fact enters the context and gets reused in subsequent turns	An earlier wrong answer ("21 leaves" instead of 12) stays in history and is repeated in every future response	Verify facts before saving; never store model guesses as ground truth
Distraction	History grows so long that the model defaults to repeating past behaviour	In an 80-message chat, the assistant keeps copying its earlier answer format even when the question type changes	Trim or summarise history beyond a defined threshold
Confusion	Irrelevant material is retrieved and gets used in the answer	Travel policy passages retrieved for a leave question; answer mixes entitlements from both policies	Retrieve fewer, better-matched passages with a higher relevance threshold
Clash	Two sources that disagree are both retrieved	Old policy v2 and new policy v3 both retrieved; model hedges or picks the wrong version	Keep only current versions; tag every document with its effective date

Source: Drew Breunig, "How Long Contexts Fail" and "How to Fix Your Context" (Jun 2025)

The "Too Many Tools" Problem

A tool is an action the model can request: check a leave balance, raise a ticket, search a database. Every tool's description occupies space in the request, whether that tool is used or not. Adding tools feels like adding capability. Past a certain point, it reduces accuracy – because the model must choose between increasingly overlapping options.

1

0–20 Tools

Clear boundaries. The model can reliably identify which tool matches which situation. Shopify calls this the comfortable range.

2

20–50 Tools

Blurred boundaries. Tool purposes begin to overlap. The model starts making suboptimal choices between similar-sounding actions.

3

50+ Tools

Multiple ways to accomplish the same thing. Reasoning about which to choose becomes unreliable. Manus found wrong action choices increased as users added extra tools.

Anthropic's practical test: if a human engineer cannot immediately say which tool fits a given situation, the model cannot either. Use that as your design bar. *Source: Shopify Engineering (Aug 2025); Breunig (Jun 2025); Manus (Jul 2025)*

Part 3 Summary: More Context Is Not Better Context

The right information, in the right order, at the right size.

Everything in the following sections has one goal: building a request that satisfies that principle. Before moving on, a summary of what the research established:

- **Position matters**

The middle of a request gets missed — critical evidence belongs at the start or end (Lost in the Middle, Stanford)

- **Length itself reduces reliability**

Independent of task difficulty, longer inputs consistently degrade every model tested (Context Rot, Chroma; RULER, NVIDIA)

- **Requirements given in pieces cause drift**

Multi-turn conversations cause a 39% average performance drop compared to complete, upfront instructions (Microsoft & Salesforce)

- **Four predictable failure modes**

Wrong, stale, conflicting, or excess content causes Poisoning, Distraction, Confusion, and Clash failures — each with a specific fix (Breunig)

- **Too many tools cause wrong actions**

Beyond approximately 20 tools, model action selection degrades measurably (Shopify, Manus, Breunig)

Four Moves Teams Use: Write, Select, Compress, Isolate

LangChain published this vocabulary in June 2025 after reviewing the design patterns used in popular agent systems and cross-referencing them with the research literature. When your engineers discuss context management, they will almost always be describing one of these four moves. The next eight slides cover each one with real examples.

WRITE

Save information outside the request so it is not lost when the conversation ends or the window fills. Includes notes, to-do lists, and memory across sessions.

"Where do we keep what we will need later?"

SELECT

Pull only the information relevant to this specific step into the request. Includes retrieval, just-in-time context, and tool selection.

"What does this step actually need?"

COMPRESS

Shrink what is already in the request – through summarisation, trimming old outputs, or replacing verbose content with short placeholders.

"What can be shortened or removed?"

ISOLATE

Split work into separate requests, each with a clean, focused context. Sub-agents return condensed summaries rather than full raw outputs.

"Should this sub-task get its own clean request?"

WRITE (1): Notes Within a Long Task

The problem

Long tasks have many steps. A decision made at step 3 – which approach to take, which file was edited, which error was encountered – may be critically needed at step 40. Without a record, the only way to preserve it is in the growing request itself, which eventually overflows or gets summarised and the detail is lost.

The solution

The agent writes notes to a file outside the request – a plan, a progress log, a list of decisions made. At later steps, the system reads the relevant notes back into the request. After a reset or failure, the agent reads its notes and continues from where it stopped.

Who does this in practice

Claude Code

Maintains a to-do list of pending tasks, updated after each step

Manus

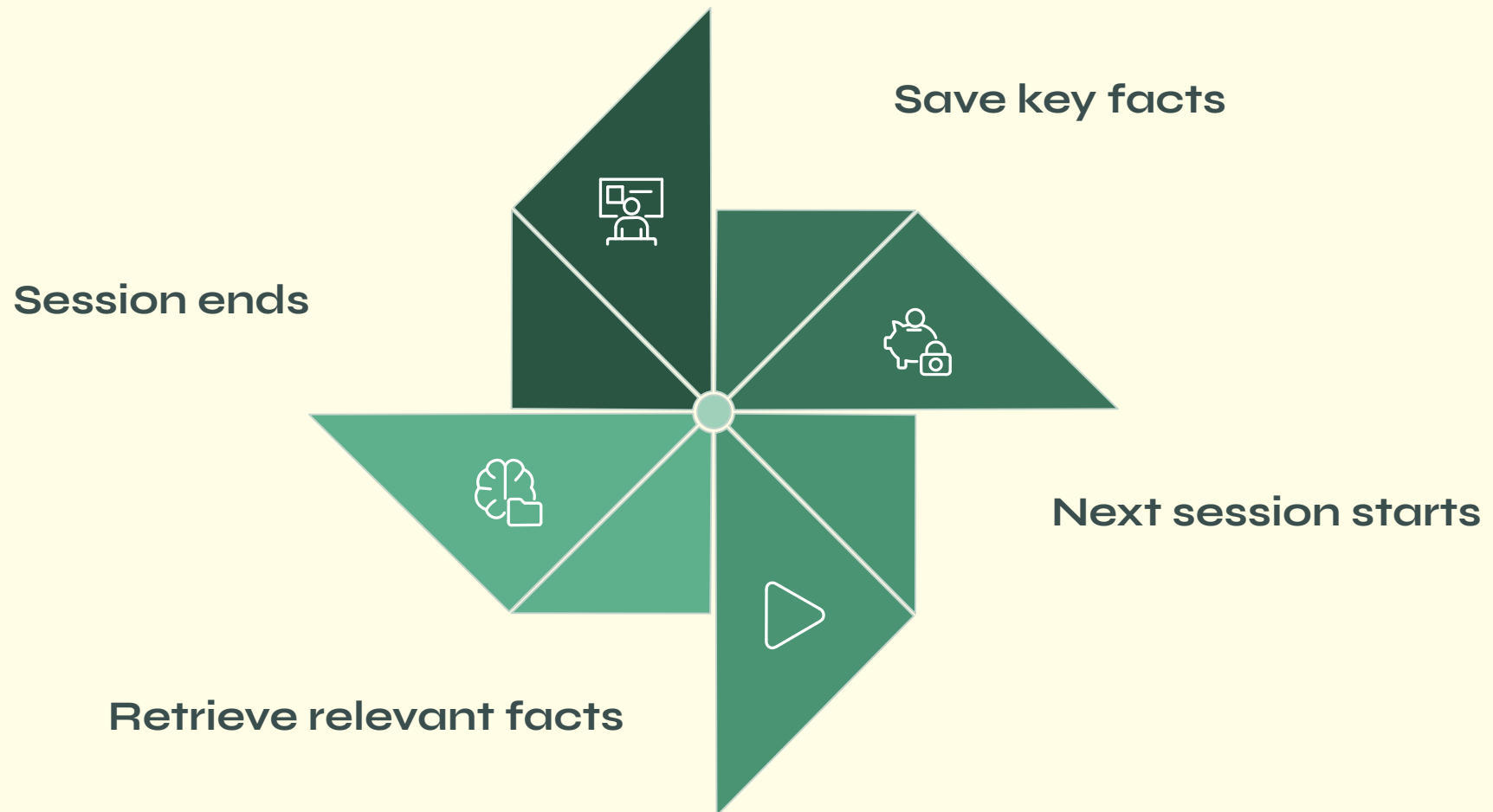
Keeps a `todo.md` file; Anthropic's Claude played Pokémon for thousands of steps by maintaining its own notes and maps

HR Onboarding Agent

Records which forms are completed, which are pending, and which the employee has flagged as questions

WRITE (2): Memory Across Sessions

Memory is the persistent storage of facts that survive after a chat session ends: user preferences, past decisions, project status, office location. Without memory, every new session starts completely cold – and the user must re-explain context the assistant already gathered weeks ago.



39%

Improvement from memory + clearing

Anthropic's internal test: a memory tool combined with automatic clearing of old tool results improved agent performance by 39% versus no management

84%

Token reduction in web-search tasks

In a 100-step web-search test, clearing old results cut token use by 84% and allowed tasks to complete that otherwise ran out of context space

⚠️ Leader question: What is saved to memory? Who can view it? How can users delete it? Memory that stores incorrect facts will repeat those facts in every future session – making it a privacy and accuracy risk, not just a convenience feature.

SELECT (1): Retrieval — Also Called RAG

RAG — Retrieval-Augmented Generation — is the most common form of the Select move and the foundation of most enterprise AI assistants. Rather than loading all company documents into every request (expensive, slow, and counterproductive), the system searches first and gives the model only the matching passages.

1 Documents are split into passages and indexed

Each policy document, FAQ, or procedure guide is divided into searchable chunks and stored in a retrieval system that can find relevant passages quickly

2 User asks → system finds best matches → 3–5 passages added

When an employee asks a question, the system retrieves the top 3–5 passages that match the query. The model sees those passages — and only those passages

3 If no passage scores above a set threshold → "I don't know"

The fallback is essential. Without a score threshold, a weak match is treated as a good match, and the model produces a confident-sounding wrong answer

Key decisions your team must make: passage size, number of passages returned, freshness filtering, and who is allowed to see which documents (permission filtering). We will see Morgan Stanley's approach to this at scale in Part 5.

SELECT (2): Load Upfront, or Fetch When Needed?

Load upfront

Gather relevant data before the model starts working. Fast, predictable, and simple to implement. Best for content that is stable and small – a short project brief, a fixed set of rules, a known user profile.

Risk: content may be stale by the time the model uses it. Can include more than is needed if the relevant subset is hard to predict in advance.

Example: Claude Code's CLAUDE.md

A short project instruction file always included in the request. It is stable, small, and needed in every step – a perfect candidate for upfront loading.

Fetch when needed ("just-in-time")

Give the agent file names, links, or saved queries. It opens content only when a specific step requires it. The agent keeps a list of where things are – not the things themselves.

Risk: slower, because each fetch is a round-trip. Requires well-designed tools for the agent to navigate. But keeps each request small and current.

Example: Claude Code for large codebases

Code files are searched and opened only when needed. A large codebase never loads all at once – only the files relevant to the current task enter the request.

Anthropic suggests the mixed approach – stable content upfront, dynamic content fetched on demand – suits regulated and professional domains such as legal, finance, and HR work. *Source: Anthropic, "Effective context engineering for AI agents" (Sep 2025)*

SELECT (3): The Right Tools and Rules at the Right Moment

The Select move applies to instructions and tool descriptions, not just documents. A main system prompt that tries to cover every special case grows until the rules contradict each other – a failure mode Shopify calls "Death by a Thousand Instructions."

Tool selection: include only relevant tools per step

Rather than loading descriptions for all available tools in every request, the system searches a catalogue and includes only the few tools relevant to the current step. LangChain reports tool choice accuracy improved approximately threefold in one study when irrelevant tools were excluded from the context.

Shopify's Just-in-Time instructions

Instead of one enormous main prompt containing every special rule, each tool's specific instructions are returned alongside that tool's result – only when the tool is actually used. Result: a shorter main prompt, fewer conflicting rules, and significantly easier maintenance as the product evolves.

HR assistant example

The rules for calculating maternity-leave entitlements are complex and specific. They appear in the request only when the maternity-leave tool is called – not in every single request the employee makes. This keeps the standard request lean and focused.

Source: LangChain (Jun 2025); Shopify Engineering, "Building production-ready agentic systems: Lessons from Shopify Sidekick" (Aug 2025)

COMPRESS (1): Summarising Long Histories

How compaction works

As a conversation nears the token limit, the system asks the model to summarise the history. The new request starts with that summary plus the most recent messages – full detail where it matters most, compressed form for older content.

Claude Code's compaction rules:

- Keep: decisions made, open bugs, key details
- Carry over: the five most recently opened files
- Drop: repeated tool outputs and intermediate results

The risks of summarisation

Summarising is irreversible. Every summary discards information – and you cannot always predict which detail will matter at step 40. That is why teams combine summaries with external notes and file references.

The "context collapse" warning

Stanford's ACE research warns against rewriting a summary again and again. Each rewrite discards more detail, and the cumulative loss is severe. Write to notes; summarise history only once before compacting further.

Anthropic's order of operations

First, verify the summary captures everything important. Only then remove the excess. This sequencing prevents discovering the loss too late.

Source: Anthropic (Sep 2025); Zhang et al., "Agentic Context Engineering" (arXiv 2510.04618, ICLR 2026)

COMPRESS (2): Trimming Old Tool Outputs

Tool outputs – search results, file contents, API responses, log data – are typically the largest component of an agent's growing request. Summarising them with AI sounds sophisticated, but research shows the simpler approach wins.

The simple method

Keep the last few tool outputs in full. Replace older ones with a short placeholder: *"8 earlier search results omitted – see notes file."* No extra model calls. Fast. Cheap. The agent's own reasoning remains fully visible.

What JetBrains Research found

This simple trimming cut cost roughly in half compared to no management, and matched or beat AI summarisation on the proportion of tasks successfully completed. AI-written summaries made agents run 13–15% longer, because they hid the signs that an agent was stuck in a loop.

When to add AI summarisation

Only where simple trimming is insufficient – for example, when a single tool output is itself extremely long (a 50-page document) and must be condensed to its key findings before the agent can proceed meaningfully.

📌 **Leader takeaway:** the simpler approach won. Try the cheap method first. Add complexity only where it is clearly necessary and has been measured against a test set. *Source: Lindenbauer et al., "The Complexity Trap" (JetBrains Research & TUM, arXiv 2508.21433, NeurIPS DL4Code 2025)*

ISOLATE (1): Sub-Agents With Their Own Clean Requests

How sub-agent isolation works

The main agent plans and coordinates. For broad tasks with independent sub-problems, it sends focused sub-tasks to separate sub-agents, each working in a fresh request with only the information it needs. Each sub-agent returns a condensed summary – typically 1,000–2,000 tokens – rather than its full working output.

Anthropic's Research feature results

A lead agent with parallel sub-agents beat a single agent by **90.2%** on internal research tests. In their analysis, the number of tokens spent explained 80% of the difference in performance.

Example task: list the board members of every IT company in the S&P 500. Splitting into parallel searches worked. A single agent searching one by one ran out of context and time.

The cost reality

A multi-agent research system costs approximately **15 times** the tokens of a normal chat. A single-agent version uses about 4 times normal. This is real money at scale.

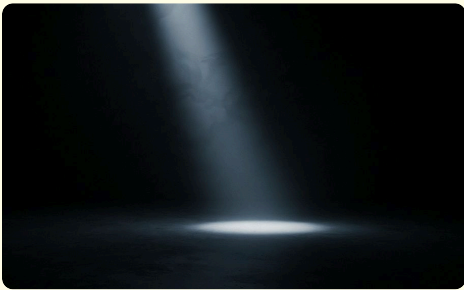
The pattern is justified only when:

- The task is broad with many parallel, independent lookups
- The answer quality improvement is worth the cost multiple
- The sub-tasks are genuinely independent (no shared state or conflicting decisions)

Source: Anthropic, "How we built our multi-agent research system" (Jun 2025)

ISOLATE (2): The Debate on When Not to Split

In June 2025, two influential posts appeared within a day of each other and seemed to contradict each other. Read together, they actually agree — and the combined lesson is the most useful design guidance available.



Cognition (Devin): "Don't Build Multi-Agents" — Jun 2025

Devin is an autonomous coding agent. Their example: sub-agents each building part of a game clone produced sections with mismatched visual styles — because each sub-agent made its own design assumptions without seeing what the others had done. **Rule 1:** every agent must see the full history of actions, not just its own instruction. **Rule 2:** never split decisions that carry hidden interdependencies. **2026 update:** one agent writes changes; additional agents contribute only analysis and review.



Anthropic: Sub-Agents Excel at Parallel, Independent Work

Looking up board members across 500 companies is inherently parallel. Each lookup is independent — no sub-agent's result affects another's. Splitting this across sub-agents gave a 90.2% improvement. The tasks are independent, so the shared-history problem does not apply.

- ❏ **Decision rule:** Split for independent research tasks that can run in parallel. Keep one agent for tightly linked, step-by-step work where decisions depend on each other. Ask your team: are our sub-tasks truly independent?

Case Study: Manus (1) – Designing for Speed and Cost

Manus is a general-purpose AI agent whose team rebuilt its framework four times while learning what reliably works at production scale. Their engineering blog is one of the most transparent accounts of context management in practice. Their primary metric is the cache hit rate: the share of each request that is identical to the previous request, allowing the provider to reuse earlier processing instead of re-reading it from scratch.

The 10x cost difference

With Claude Sonnet: reused (cached) input costs \$0.30 per million tokens. New input costs \$3.00. A high cache hit rate is therefore not a technical curiosity – it is a 10x difference in operating cost.

Keep the start fixed

Caching works only when the beginning of the request is byte-for-byte identical to the previous request. A clock timestamp near the top – even one changed word – forces full reprocessing at full price. Manus removed all dynamic content from the request header.

Only append, never edit

New content is added to the end of the request only. Earlier parts are never modified mid-task. Tools are not added or removed once a task has begun. This discipline is what makes the cache hit rate achievable.

Leader question to ask your team: "What is our cache hit rate?" If no one knows, the cost optimisation has not started. *Source: Manus (Jul 2025)*

Case Study: Manus (2) – Memory, Focus, and Error Visibility

Beyond caching, Manus developed four additional operational rules that each map directly to a research finding from Part 3. This alignment is not coincidental – practitioners who encounter the same problems in production tend to arrive at the same solutions the researchers identified in controlled experiments.

Files as memory – references, not content

Large web pages, PDFs, and documents are saved as files. The request contains only the link or file path. The agent reopens files when it needs them, so no content is permanently lost and the request stays lean. Maps to: Compress without losing the ability to recover.

Task-list rewriting at the end

After each step, the agent rewrites its to-do list and places it at the *end* of the request – the high-attention position. Across approximately 50 tool calls, the goal stays in the sharpest focus zone. Maps to: Lost in the Middle.

Keep errors visible

Failed actions and error messages stay in the request. The model can see what it already tried and failed, so it does not repeat the same mistake. Maps to: Poisoning (preventing repeated errors from becoming repeated actions).

Avoid repetition in structure

When reviewing 20 similar items, identical-looking step structures made the agent repeat the same analytical pattern even when it no longer fit. Small formatting variations between steps broke the pattern. Maps to: Distraction.

Case Study: Anthropic Claude Code – A Coding Agent Using All Four Moves

Claude Code is the clearest compact example of all four moves deployed together in one production system. It is also the system Anthropic uses internally and therefore represents their most tested thinking about context management.



WRITE

Maintains a to-do list for long tasks. When sub-agents investigate a side question, they return a condensed summary – not their full working context – to the main agent.



SELECT

CLAUDE.md (a short project instruction file) is always included upfront. All other code files are searched and opened on demand. A large codebase never loads all at once – only what the current step needs enters the request.



COMPRESS

Automatic compaction triggers as the window fills. The system keeps decisions, open bugs, and key details; drops repeated tool outputs; and carries over the five most recently opened files.



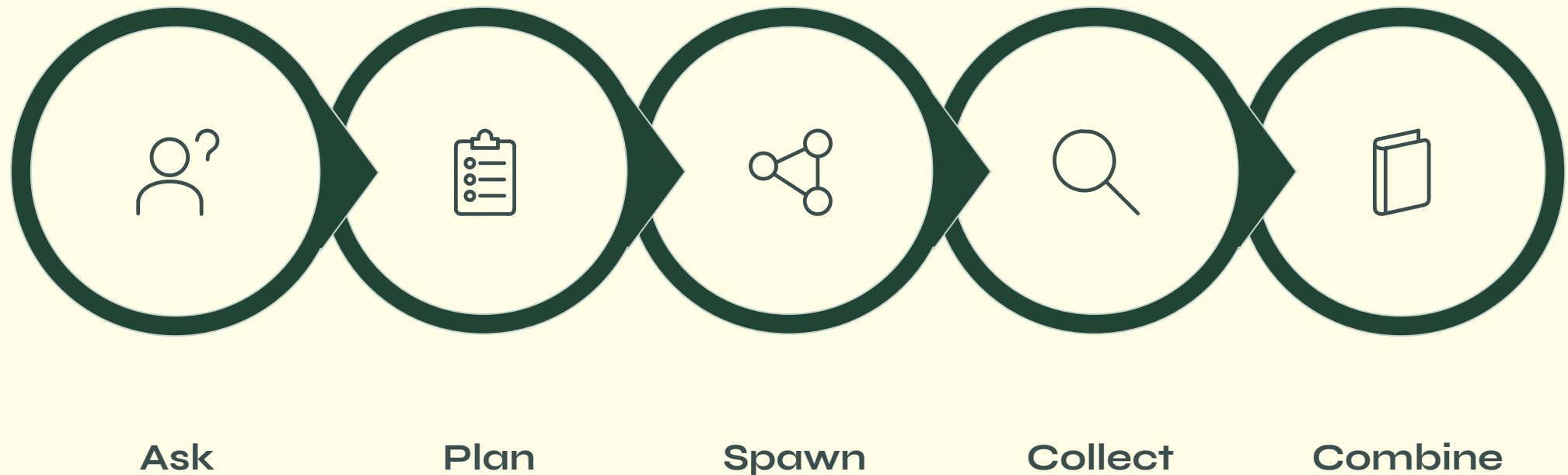
ISOLATE

Side investigations – "does this library have the method we need?" – are handed to sub-agents that return brief findings. The main agent's context stays focused on the primary task.

Anthropic's standing advice to teams: **do the simplest thing that works**. Claude Code reached this design through iteration, not upfront architecture.

Source: Anthropic (Sep 2025)

Case Study: Anthropic Research – Multi-Agent at Scale



What made this necessary

Consider this task: list every board member of every IT company in the S&P 500. A single agent searching sequentially would run out of context space and time before completing. Splitting across parallel sub-agents – each responsible for a slice of companies – made the task feasible.

The measured results

The multi-agent system outperformed a single agent by **90.2%** on internal research quality tests. Token spending explained 80% of the performance difference – more tokens spent on parallel investigation directly translated to better answers. Cost: approximately 15x a normal chat interaction.

Source: Anthropic, "How we built our multi-agent research system" (Jun 2025)

Case Study: Cognition (Devin) – One Thread First

Devin is an autonomous coding agent designed to build software independently. Cognition's engineering lessons are particularly valuable because they come from a domain – software construction – where consistency across components is non-negotiable.



The game-clone failure

Sub-agents each building a section of a game clone produced sections with visually mismatched styles. Each sub-agent made its own aesthetic decisions without knowing what the others had done. The output was technically functional but not a coherent product.



Rule 1: full history, not just the instruction

Every agent should see the complete history of actions taken so far, not just its individual task description. Without full history, every hidden assumption each sub-agent makes is an opportunity for conflict with another sub-agent's assumptions.



The 2026 update: writers and reviewers

One agent makes changes. Additional agents contribute analysis and review only. This preserves the benefit of multiple perspectives while preventing conflicting writes to shared state.

Shopify concurs: avoid multi-agent designs early in a project's life. A single agent handles more than you expect, and adding coordination complexity before it is genuinely necessary slows everything down. *Source: Cognition (Jun 2025; Apr 2026); Shopify Engineering (Aug 2025)*

Case Study: Shopify Sidekick — An Assistant for Merchants

Sidekick helps store owners analyse customers, fill product forms, and navigate the Shopify admin — all through a chat interface. As the product grew, Shopify encountered the context problem that all ambitious AI products hit: the main prompt became unmanageable.

"Death by a Thousand Instructions"

As Sidekick's capabilities expanded to cover more situations, every special case got added to the main system prompt. Past 50 tools, the prompt became a long list of conflicting rules that the model could not reconcile. Rules for edge cases in one domain began interfering with standard behaviour in another.

The fix: Just-in-Time instructions

Each tool's specific rules now come back with that tool's result — only when the tool is invoked. The main prompt stays short and stable. Tool-specific guidance arrives exactly when it is relevant and nowhere else.

Quality measurement: AI judges calibrated to humans

Shopify built AI judges to evaluate Sidekick's answers at scale. They tuned these judges until their agreement with human reviewers reached a Cohen's kappa of 0.61 — close to the 0.69 that human experts reach when reviewing each other's work. This calibration step is essential: an uncalibrated judge can make things worse.

The safety layer: preview before save

Before any change is made to a merchant's store, Sidekick shows a preview. The merchant approves before anything is saved. This design choice — which your team should replicate in any assistant that modifies records — ensures the human stays the decision-maker.

Source: Shopify Engineering (Aug 2025)

Case Study: Google ADK – The Request Is Built Fresh From Stored Data

Google's Agent Development Kit formalises a principle that the best-performing teams arrive at through experience: storage and the model request are two entirely separate design problems, and conflating them causes the most common scaling failures.

What is stored (persistent)

Sessions: the chat log and current task state.

Memory: long-term facts about the user or project. **Artifacts:** files such as CSVs, PDFs, reports, and transcripts. These live in proper storage systems, not in the model request.

What is built fresh (per request)

Before each model call, a sequence of processing steps filters, summarises, and assembles the relevant pieces from stored data into a focused request. Engineers decide, as a normal system design task, where each filtering and summarising step happens.

The warning: "context dumping"

Pasting a 5 MB CSV directly into the chat. Returning a huge API response as raw text. Google explicitly warns against both. Store it as an artifact; refer to it by name or query only the fields you need. The model does not need to read the entire file to answer a specific question about it.

📌 **Leader takeaway:** your team needs to design both a storage system and a request assembly system. These are different engineering problems with different owners, update cycles, and quality criteria. *Source: Google Developers Blog, "Architecting efficient context-aware multi-agent framework for production" (Dec 2025)*

Case Study: Anthropic "Code Execution with MCP" – Keep Bulk Data Out of the Request

The old flow – data passes through the model

Agent calls Tool A → full result enters the request → model reads and copies it → model calls Tool B with the copied data → full data passes through the request again.

Example: copying a meeting transcript from Google Drive into Salesforce using the old flow.

Cost: approximately 150,000 tokens.

The new flow – data moves directly

Model writes a short programme → programme moves data from Tool A to Tool B directly → only a short confirmation result returns to the model.

Cost: approximately 2,000 tokens. A 98.7% reduction.

MCP: Model Context Protocol

MCP is an open standard for connecting AI models to tools and data sources. The key insight behind the new flow: the model only needs to know that the transcript moved successfully – not to read every word of it. The model writes the logic; the programme executes it.

What this enables

Dramatically lower cost, faster responses, and – critically – sensitive data never passes through the model at all. A document that moves directly from one system to another is not read by the AI, which matters enormously for compliance and privacy in regulated industries.

The trade-off

The engineering team must run model-generated code in a secure, isolated environment. This is a non-trivial infrastructure requirement, but one that pays off quickly at scale.

Source: Anthropic Engineering, "Code execution with MCP" (Nov 2025)

Case Study: Morgan Stanley – Retrieval in a Regulated Bank

The Morgan Stanley AI @ Morgan Stanley Assistant demonstrates what the Select move looks like at enterprise scale, inside a tightly regulated environment where incorrect information has direct financial and compliance consequences.

100K

Documents indexed

Grew from answering a fixed set of approximately 7,000 questions to effectively any question across 100,000 proprietary research documents

98%+

Advisor team adoption

Over 98% of advisor teams actively use the assistant – a rare adoption figure that reflects genuine utility rather than mandated usage

80%

Document access rate

Access to the firm's research library rose from 20% to 80% of content being actively used – a fourfold improvement in knowledge utilisation

How they achieved it: steady, measured improvement of retrieval quality – evaluated through formal testing including summarisation and translation tasks – with human advisor review of AI outputs before they reach clients. No single breakthrough. Continuous iteration against a measured baseline.

Lesson for leaders: in regulated work, the context system is: retrieval quality + evaluation rigour + human review. Clever prompt wording explains almost none of the gains. *Source: OpenAI customer story, "Morgan Stanley uses AI evals to shape the future of financial services"; Morgan Stanley press release (Jun 2024)*

Research Check: Do Instruction Files Actually Help? — ETH Zurich, Feb 2026

Files like AGENTS.md and CLAUDE.md have been widely adopted as a best practice for giving coding agents project-level instructions. ETH Zurich ran a rigorous test of whether this actually works.

The experimental design

4 coding agents tested on real GitHub issues across three conditions:

- No instruction file (baseline)
- AI-generated instruction file
- Developer-written instruction file

The results

AI-generated files: slightly lower success rate (approximately 3% worse) and over 20% higher cost. The files added noise and incomplete context that agents had to work around.

Developer-written files: approximately 4% higher success — but still up to 19% higher cost. Agents followed the files closely, so unnecessary instructions made tasks harder and longer.

What worked and what did not

General project overviews: did not help. Agents already inferred the project structure.

Specific, unusual requirements that the agent could not infer: helped meaningfully. Custom test commands, non-standard build processes, organisation-specific conventions.

Leader takeaway

A widely recommended practice turned out to help only marginally and cost more. This keeps us honest: every piece of added context should be tested against a baseline, not assumed to help because it sounds reasonable.

Source: Gloaguen et al., "Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?" (ETH Zurich, arXiv 2602.11988, Feb 2026)

Use Case: Bank Customer Support Assistant

A bank customer support assistant must balance helpfulness with strict data protection requirements. The context system is simultaneously an AI design problem and a regulatory compliance problem – and the most important design decisions are about what stays *out*, not what goes in.

Context Component	What It Contains	Design Rule
Fixed context	Tone rules, compliance disclaimers, refusal rules (e.g., no investment advice, no legal opinions)	Stable, small, cached. Reviewed by compliance before any change.
Customer data	Only the product fields needed for this question – fetched via tool, not the full account profile	Fetch only the fields the question actually requires. Never load the full record.
Knowledge	Top matching FAQ and policy passages for this specific question	Retrieve fewer, higher-scoring passages. Quality threshold enforced.
Must stay out	Other customers' data, full account numbers, unapproved content, investment recommendations	Filter before retrieval. Permission checked at the user level, not the document level.

Moves used: Select (only customer fields needed for this question), Compress (summary when chat exceeds threshold), Just-in-Time rules for complaints and escalation flows. The most important design line: *what must stay out* is as carefully specified as what goes in.

Use Case: Insurance Claims Assistant

An insurance claims assistant must help a claims officer verify a claim against the policy quickly and accurately – while handling complex documents, multiple policy versions, and strict regulatory requirements.



Claim Received

Extract & Identify

Retrieve Clauses

Assess & Draft

WRITE

A case-notes file records extracted fields, retrieved clauses, and draft decisions. The officer can review the full audit trail.

SELECT

Clause retrieval filtered by product type and policy start date. Hospital bills and reports stored as files; only extracted fields enter the request – not 50-page PDFs.

ISOLATE

A separate sub-task reads long medical documents and returns a structured summary: diagnosis, dates, procedures. Only the summary enters the main request.

The date filter is the most critical design decision. Multiple policy wordings co-exist in the archive. Retrieving the wrong version – applying a 2022 clause to a 2024 policy – is the most common cause of incorrect claims decisions in AI-assisted workflows. Filter by policy start date before retrieval, not after.

Use Case: Internal Engineering Assistant

Goal and scope

Help developers find and fix bugs, add features, write tests, and navigate the company codebase – without needing to read the entire codebase into every request or start each session from scratch.

Always included (WRITE + SELECT)

A short project file containing build commands, test procedures, and style rules. Kept deliberately minimal – per the ETH Zurich finding that general overviews add cost without adding accuracy. Only unusual, non-standard requirements go here.

Opened on demand (SELECT)

Code files are found by search and opened only when the current task requires them. Test output is trimmed to the most recent runs; full historical logs are stored as files and referenced by name.

Long tasks (WRITE + COMPRESS)

A to-do file tracks progress across a multi-step task. Compaction triggers when the window reaches approximately 70% of capacity – keeping decisions and open issues, dropping repeated tool outputs.

Independent investigations (ISOLATE)

Checking whether a library has a needed method, or reviewing a tangential area of code, goes to a sub-agent that returns a brief finding. The main task context stays focused.

Human approval (Governance)

All changes are proposed as reviewable pull requests. No code is merged without a human engineer's approval. The assistant prepares; the engineer decides. This is non-negotiable in any system that modifies production artefacts.

A 7-Step Context Management Framework

This framework applies whether you are designing a new assistant from scratch or auditing one already in production. Steps 1–3 are planning. Steps 4–5 are design. Steps 6–7 keep the system reliable after launch.

01 Define the job What the assistant must answer, must refuse, and must hand to a human. Be specific about the boundaries – ambiguity here causes all four failure modes downstream.	02 List every context source Owner, update frequency, freshness requirement, and who is allowed to see it. A source with no owner will go stale and cause clash failures.	03 Design the request layout and token budget Fixed parts first, changing parts next, key reminders last. Set a token budget for each section so compression and selection rules have concrete targets.
04 Set selection rules For documents: number of passages, minimum match score, current versions only, permissions. For tools: maximum per step, one clear purpose per tool, trimmed results.	05 Set memory and compression rules What may be saved, what must never be saved, how users view and delete it. When compression triggers, what is preserved, what is summarised, what is dropped.	06 Evaluate with a fixed test set Before and after every change. Build a golden set of real questions with correct answers reviewed by domain experts. Track correct answers, correct citations, refusals, cost, and cache hit rate.
07 Govern and improve continuously Access control, privacy filtering, version history for instructions and retrieval settings, weekly failure review, improvement loop back to steps 4–5.		

Steps 1–2: Define the Job and List the Sources

Most context engineering problems begin before any engineering. If no one owns a document source, or if its update process is unclear, that source will eventually go stale – and the resulting clash failure will be invisible until a user receives a wrong answer.

HR Assistant – Job Definition

✓ Must Answer Leave entitlements, travel reimbursement rules, benefits eligibility, public holiday schedules, carry-forward rules by location	▪ Must Refuse Salary negotiations, performance disputes, legal questions, comparisons between individual employees' packages	⇒ Must Hand to Human Grievances, harassment reports, medical leave disputes, any question the assistant cannot answer with a cited policy section
---	--	---

Source Registry

Source	HR Assistant Example	Owner	Update Frequency	Who May See
Policy documents	Leave, travel, benefits policies	HR policy team	Each revision	All employees
Employee data	Leave balance, location, grade	HR system	Real time	That employee only
Chat history	Current conversation	Application	Every message	That employee only
Saved memory	Preferred language, office location	Application	When user changes it	That employee only

Step 3: Request Layout and Token Budget

A token budget turns a vague goal – "keep requests small" – into concrete numbers that engineers can test, measure, and tune against the evaluation set. The order of elements within the request follows directly from the research findings in Part 3.

The ordering principle

Fixed parts first: system instructions and tool list. These are stable, cacheable (Manus cache-hit principle), and provide the model's operating framework.

Changing parts next: memory facts, retrieved documents, conversation history. These vary by request but are not the most critical item to attend to.

User question and key reminders last: the end of the request receives the highest model attention (Lost in the Middle). The question and any critical constraints belong here.

Illustrative HR assistant token budget

Part	Token Budget
System instructions	1,500
Tool descriptions (5 tools)	1,500
Memory facts	300
Retrieved passages (top 4)	3,000
Conversation history	2,500
User question + reminders	500
Total	~9,300 – far below window, on purpose

When history exceeds 2,500 tokens, compression triggers. When retrieval returns more than 3,000 tokens, lowest-scoring passages are dropped. These are examples; your team should tune against the evaluation set.

Steps 4–5: Selection, Memory, and Compression Rules

Each rule below traces directly to a case study or research finding from this session. Ask your team to write these rules down as a one-page design document before any assistant goes to production.

Document selection rules

Number of passages to retrieve (start with 3–5), minimum relevance score threshold (below which the assistant says it does not know), current versions only with effective-date tagging, filtered by user permissions before retrieval. *Traces to: RAG, Morgan Stanley, Clash failure*

Tool rules

Stay under approximately 20 tools per step. One clear, non-overlapping purpose per tool. Tool results trimmed to only the fields the model needs for this step – not the full API response. *Traces to: Shopify, Breunig, Manus*

Memory rules

What may be saved (preferences, location, language). What must never be saved (medical data, financial details, model guesses). How users view, correct, and delete their memory. *Traces to: Anthropic 39% improvement, GDPR-style governance*

Compression order

1. Trim old tool outputs first (simple, cheap, no model calls).
2. Summarise history only past the budget threshold (70–80% of budget).
3. Never compress away decisions, open issues, or identifiers.
4. Always keep error messages visible. *Traces to: JetBrains, Manus, Claude Code compaction rules*

Step 6: Evaluate Before and After Every Change

The gap between monitoring and evaluation is where most teams lose ground. Monitoring tells you what the system did. Evaluation tells you whether it was right.

57%

Agents in production

LangChain survey of 1,300+ practitioners (late 2025): 57% have agents running in production environments

89%

Monitor their agents

89% of teams with production agents monitor what their agents do – they can see the actions taken

52%

Run formal evaluations

Only 52% run formal evaluations against a test set – meaning nearly half cannot tell whether a change improved or degraded their system

32%

Cite quality as top barrier

Quality is cited as the top barrier to wider agent deployment – by teams that often lack the evaluation infrastructure to improve it systematically

What your golden test set must include

Real questions drawn from actual user queries, with correct answers verified by domain experts. AI judges calibrated against human reviewers (Shopify's method: tune until $\kappa \approx 0.6$). Track per change: correct answer rate, correct citation rate, appropriate refusal rate, cost per task, response time, and cache hit rate.

The non-negotiable rule

No change to instructions, retrieval settings, or memory rules goes live without re-running the full evaluation set. Morgan Stanley, Shopify, and ETH Zurich all show the same pattern: measured changes improve systems; assumed improvements often do not.

Step 7: Govern and Improve Continuously

Governance is where programme managers and team leads have the most direct influence. The engineering team builds the system; leaders define what is allowed in it and ensure the improvement loop runs consistently.



Access control

A request must contain only what that specific user is authorised to see. Filter before retrieval – checking permissions after the fact is insufficient and potentially exposes unauthorised data to the model even if it does not appear in the final answer.



Privacy

Define which personal data may enter a request and what must be masked or excluded entirely. This policy belongs in a written document, not in an individual engineer's memory. Review it when regulations change.



Version history

Store system instructions, tool descriptions, and retrieval settings like code – with a dated record of every change and who approved it. When a regression occurs, you need to know exactly what changed and when.



Improvement loop

Review failures weekly. Update instructions or retrieval settings. Re-run the evaluation set. ACE research (Stanford, ICLR 2026) shows that maintaining a "playbook" of small, itemised lessons from failures – rather than rewriting everything – produced +10.6% on agent tasks and +8.6% on finance tasks.

Source: Zhang et al., "Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models" (arXiv 2510.04618, ICLR 2026)

Ten Questions to Ask Your Team

These questions are designed to surface context problems that teams often do not know they have. Question 1 is the most powerful: many teams have never printed out a complete request, and doing so typically reveals duplicated instructions, stale documents, and unused tools within minutes.

1

Show me one full request the model receives. What is in it?

Ask for an actual, complete request from your production system – not a diagram or description of what *should* be in it.

2

Where does each part come from, and who owns it?

Every component of the request should have a named owner and a documented update process.

3

How many tokens does a typical request use, and what does it cost?

If the team does not know the cost per request, the cost optimisation has not begun.

4

What happens when no relevant document is found?

The fallback behaviour is as important as the happy path. "I don't know" should be reliable, specific, and not a confident hallucination.

5

How do we stop old or conflicting document versions from being retrieved?

Clash failure is the most common cause of subtly wrong answers in document-based assistants.

6

How many tools does the model see at once?

The answer should be under approximately 20 for any given step. If it is higher, ask why.

7

What happens when a conversation or task gets long?

Is there a compression rule? When does it trigger? What is preserved and what is dropped?

8

What does the system remember across sessions, and can users delete it?

Memory without a deletion mechanism is a privacy risk. Memory that stores incorrect facts repeats those facts in every future session.

9

What is our evaluation set, and what were the latest results?

If there is no golden test set, quality improvements are guesswork.

10

Can the model ever see data the user is not allowed to see?

Permission filtering must happen before retrieval, not after. This question should have a definitive, documented answer.

Recap and One-Page Checklist

Five things to remember

- **Prompt engineering is not dead**

It is one part of a larger discipline. A well-crafted instruction still matters – but it is no longer sufficient on its own.

- **The model knows only what is in the request**

Your system builds that request every time, for every user, across every session. That is the design problem.

- **Longer requests cost more, run slower, and are less accurate**

The right amount of context – not the most context – is the goal.

- **Four moves: Write, Select, Compress, Isolate**

Every technique in this field is one of these four. Use them as your vocabulary with your engineering team.

- **An AI answer is only as good as the request your system builds**

The teams that succeed treat every request as a designed product, with owners, budgets, rules, and tests.

Pre-launch checklist

- ☐ Owner assigned for every context source
- ☐ Token budget defined per request section
- ☐ Fixed start of request for caching
- ☐ Approximately 20 tools or fewer per step
- ☐ Compression rule written down and tested
- ☐ Memory policy documented with deletion mechanism
- ☐ Golden evaluation set built and baselined
- ☐ Permission filtering confirmed before retrieval
- ☐ Fallback behaviour defined and tested
- ☐ Version history in place for instructions and retrieval settings

References

All sources cited in this presentation. Where possible, follow the links to read the original engineering guides and research papers — they are written for practitioners and are more accessible than typical academic publications.

Anthropic — Core context engineering guides

[Effective context engineering for AI agents \(Sep 2025\)](#) · [How we built our multi-agent research system \(Jun 2025\)](#) · [Managing context on the Claude Developer Platform](#) · [Code execution with MCP \(Nov 2025\)](#)

Industry engineering guides

[Manus — Context Engineering for AI Agents \(Jul 2025\)](#) · [LangChain — Context Engineering for Agents \(Jun 2025\)](#) · [LangChain — State of Agent Engineering](#) · [Shopify Engineering — Lessons from Shopify Sidekick \(Aug 2025\)](#) · [Google ADK — Context-aware multi-agent framework \(Dec 2025\)](#)

Research papers

[Liu et al. — Lost in the Middle \(TACL 2024\)](#) · [Hsieh et al. — RULER \(NVIDIA, COLM 2024\)](#) · [Laban et al. — LLMs Get Lost in Multi-Turn Conversation \(May 2025\)](#) · [Lindenbauer et al. — The Complexity Trap \(JetBrains, NeurIPS 2025\)](#) · [Gloaguen et al. — Evaluating AGENTS.md \(ETH Zurich, Feb 2026\)](#) · [Zhang et al. — Agentic Context Engineering \(ICLR 2026\)](#)

Practitioner essays and industry reports

[Chroma — Context Rot \(Jul 2025\)](#) · [Drew Breunig — How Long Contexts Fail \(Jun 2025\)](#) · [Cognition — Don't Build Multi-Agents \(Jun 2025\)](#) · [OpenAI — Morgan Stanley customer story](#) · [Gartner — Lead the Shift to Context Engineering \(Jul 2025\)](#)